แผนการเรียนรายวิชาวิทยาการคำนวณ/คอมพิวเตอร์

ระดับชั้น: ประถมศึกษา
ระยะเวลา: 2 ชั่วโมง
วิชา: วิทยาการคำนวณ/คอมพิวเตอร์
หน่วยการเรียนรู้ : การเขียนคำสั่งฟังก์ชันพื้นฐานด้วยโปรแกรม Python

---

มาตรฐานการเรียนรู้และตัวชี้วัด

ว 4.1 เข้าใจและใช้เทคโนโลยีสารสนเทศและการสื่อสารในการสืบค้นข้อมูล การเรียนรู้ การสื่อสาร การแก้ปัญหา

ว 1.1 เข้าใจกระบวนการทางวิทยาศาสตร์และสามารถนำไปใช้ในการแก้ปัญหา

ง 2.1 มีทักษะการใช้เทคโนโลยี การคิดเชิงออกแบบ และกระบวนการทำงาน

ง 4.1 เข้าใจ เห็นคุณค่า และใช้กระบวนการเทคโนโลยีสารสนเทศในการสืบค้นข้อมูล การเรียนรู้ การสื่อสาร การแก้ปัญหา และการทำงานอาชีพอย่างมีประสิทธิภาพ ประสิทธิผล และมีคุณธรรม

ตัวชี้วัด

1. เขียนโปรแกรมแสดงผลบนจอ Dot Matrix ได้
2. เขียนโปรแกรมควบคุมไฟ RGB บน CoreX ให้กระพริบได้
3. เขียนโปรแกรมวัดระยะทางด้วยเซ็นเซอร์อัลตราโซนิกได้
4. เขียนโปรแกรมรับค่าจากปุ่ม A และ B ได้
5. เขียนโปรแกรมให้ WonderCam จดจำสีได้
6. เขียนโปรแกรมให้แขนหุ่นยนต์จับและวางบล็อคตามสีได้

จุดประสงค์การเรียนรู้

ด้านความรู้ (Knowledge - K)

1. อธิบายหน้าที่ของเซ็นเซอร์และอุปกรณ์ต่างๆ บน xArm AI ได้
2. บอกคำสั่งพื้นฐานในการควบคุมแต่ละอุปกรณ์ได้
3. เข้าใจการทำงานของเซ็นเซอร์สีและอัลตราโซนิกได้

ด้านทักษะ (Skills - S)

1. เขียนโปรแกรมควบคุมจอ Dot Matrix แสดงตัวอักษรและรูปได้
2. เขียนโปรแกรมควบคุมไฟ RGB ให้เปลี่ยนสีได้
3. ใช้เซ็นเซอร์อัลตราโซนิกวัดระยะทางได้
4. ใช้ปุ่มควบคุมการทำงานของโปรแกรมได้
5. ประยุกต์ใช้เซ็นเซอร์สีในการแยกบล็อคตามสีได้

ด้านคุณลักษณะ (Attributes - A)

1. มีความรอบคอบในการเขียนโปรแกรมและทดสอบ
2. มีความอดทนในการแก้ไขข้อผิดพลาด
3. ร่วมมือกันทำงานเป็นกลุ่ม
4. มีความคิดสร้างสรรค์ในการประยุกต์ใช้งาน

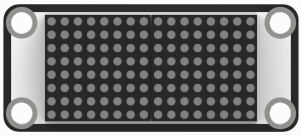
คาบเรียนที่ 17-18 : หลักการทำงานพื้นฐาน

1. จอแสดงผลแบบดอทเมทริกซ์

1.1. ข้อมูลอุปกรณ์

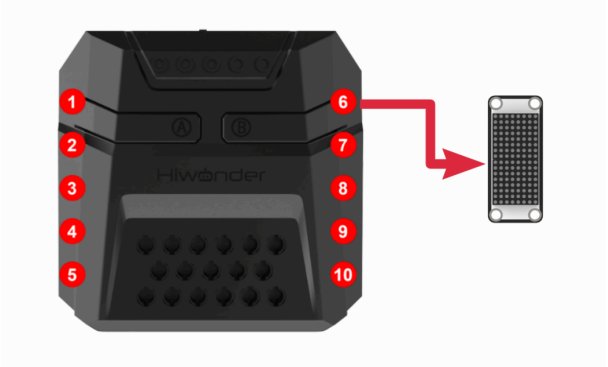
โมดูลจอแสดงผล LED Dot Matrix มีคุณสมบัติเด่นคือ ความสว่างสูง แสดงผลไม่กระพริบ ต่อสายง่าย และสามารถแสดงตัวเลข ข้อความ รูปแบบ และเนื้อหาอื่นๆ ได้

โมดูลนี้ประกอบด้วยไฟ LED สีแดงขนาด 8x8 สองชิ้น และถูกควบคุมโดยชิปไดรเวอร์ TM640B ซึ่งช่วยให้สามารถควบคุมจอแสดงผลของ Dot Matrix ได้



1.2. การเชื่อมต่อ

เชื่อมต่อ Dot Matrix เข้ากับพอร์ตหมายเลข 6 บนตัวควบคุม CoreX



1.3. การวิเคราะห์โปรแกรม Python

(1) โหลดไลบรารี:

ไลบรารี HiWonder ประกอบด้วยไลบรารีเซ็นเซอร์และฟังก์ชันสัญญาณเตือนแรงดันต่ำ

ไลบรารี Time ใช้สำหรับการดำเนินการที่เกี่ยวข้องกับเวลา

```
import Hiwonder
import time
```

(2) สร้างอินสแตนซ์ของ Digitaltube จากไลบรารี Hiwonder และผูกเข้ากับพอร์ต IO 6 หลังจากเข้าสู่ฟังก์ชันหลัก แขนหุ่นยนต์จะทำกลุ่มการกระทำ 0 หนึ่งครั้งและปรับไป

ยังท่าทางเริ่มต้น

```
# initialize variables
digitalTube_6 = Hiwonder.Digitaltube(Hiwonder.Port(6))


def start_main():
  global digitalTube_6

  LSC.runActionGroup(0,1)
```

(3) ถัดไป เรียกเมธอดสมาชิกของออบเจ็กต์โมดูลดอทเมทริกซ์เพื่อแสดงสตริงที่ระบุที่ตำแหน่งที่กำหนดบนเมทริกซ์

ฟังก์ชันนี้จะทำการ map อักขระแต่ละตัวในสตริงไปยัง LED ที่เกี่ยวข้องบน Dot matrix โดยแสดงสถานะเปิด/ปิดของอักขระนั้ จากนั้นข้อมูลนี้จะถูกส่งไปยังโมดูลเพื่อควบคุมการแสดงผล

```
  while True:
    digitalTube_6.drawStr((3,0,'Hi'))
    time.sleep(1)
    digitalTube_6.drawStr((6,0,'I'))
    time.sleep(1)
    digitalTube_6.drawStr((2,0,'AM'))
    time.sleep(1)
```

(4) สุดท้าย ใช้ซอฟต์แวร์สร้างฟอนต์เพื่อสร้างรูปแบบ “ **xarm** ” และส่งออกข้อมูลการแสดงผลดอทเมทริกซ์ที่เกี่ยวข้อง จากนั้นข้อมูลนี้จะถูกส่งไปยังโมดูลดอทเมทริกซ์โดยตรงผ่านฟังก์ชัน drawBitMap เพื่อควบคุมการแสดงผล

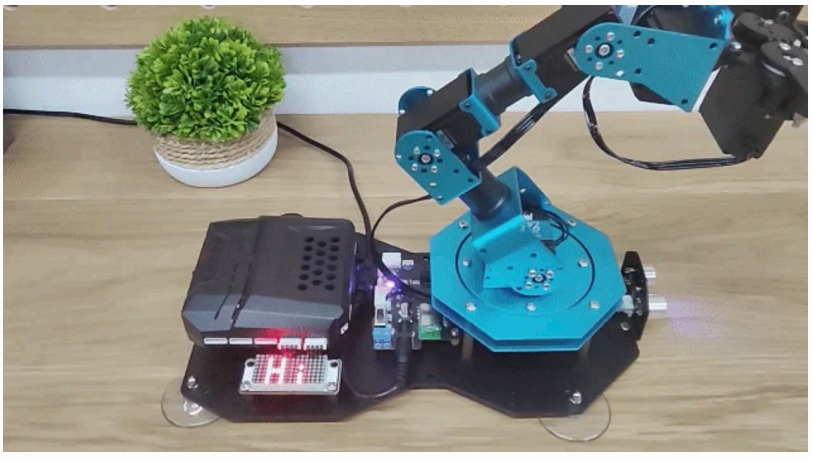
```
    digitalTube_6.drawBitMap((0x50,0x20,0x50,0x0,0x78,0x14,0x78,0x0,0x78,0x10,0x8,0x70,0x8,0x70,0x8,0x70))
    time.sleep(3)
```

(5) ปิดด้วย

```
Hiwonder.startMain(start_main)
```

1.4. ผลลัพธ์

โมดูล LED ดอทเมทริกซ์แสดงข้อความ “ Hi I AM xArm ”



2. คำสั่งกระพริบของไฟ RGB

2.1. ข้อมูลอุปกรณ์

ไฟ RGB นั้นอยู่บนพื้นฐานของหลักการของแสง ประกอบด้วยหลอด สีแดง เขียว และน้ำเงิน สามารถนำมาผสมกันเพื่อสร้างแสงสีต่างๆ ได้ ตัวควบคุม CoreX มีไฟ LED RGB ในตัว 6 ดวง



2.2. การเชื่อมต่อ

ไฟ RGB จะอยู่ในตัว CoreX ไม่ต้องติดตั้งเพิ่ม

2.3. การวิเคราะห์โปรแแกรม Python

(1) โหลดไลบรารี

ไลบรารี Hiwonder ประกอบด้วยไลบรารีเซ็นเซอร์ สัญญาณเตือนแรงดันต่ำ ฯลฯ

ไลบรารี Time ใช้สำหรับฟังก์ชันที่เกี่ยวข้องกับเวลา

และไลบรารี LSC จัดการการสื่อสารระดับต่ำระหว่างตัวควบคุมหลักและบอร์ดควบคุมเซอร์โว

```
1  import Hiwonder
2  import time
3  from Hiwonder import LSC
```

(2) ตัวแปรหน่วงเวลาถูกสร้างขึ้นเพื่อควบคุมความเร็วการวิ่งของไฟ ในฟังก์ชันหลัก ให้ตั้งค่าการหน่วงเวลานี้เป็น 0.1 วินาที เมื่อเปิดเครื่อง แขนหุ่นยนต์จะดำเนินการกลุ่มการกระทำ 0 และกลับไปยังตำแหน่งเริ่มต้น

```
5  # initialize variables
6  delay_time = 0
7
8
9  LSC.runActionGroup(0,1)
```

```
14 def start_main():
15   global delay_time
16
17   delay_time = 0.1
```

(3) ต่อไป คุณต้องเรียกฟังก์ชัน setItem ของเมธอด Neopixel_onboard ของออบเจ็กต์โมดูลไฟ RGB ในไลบรารี Hiwonder เพื่อควบคุมการเปิดและปิดไฟ RGB กระบวนการนี้คล้ายกันมากสำหรับสวิตช์ไฟแต่ละตัว นี่คือตัวอย่างหนึ่งของกระบวนการสลับ:

ขั้นแรก ตัวควบคุมจะตั้งค่าไฟ RGB ที่มี ID 1 ให้แสดงสีแดงโดยการตั้งค่าค่า RGB เป็น 255:0:0 จากนั้นจะเรียกฟังก์ชันเขียนเพื่อส่งข้อมูล และไฟ RGB ที่มี ID 1 จะเปิดขึ้น

พร้อมกับแสงสีแดง ไฟ RGB จะมีหมายเลขเริ่มต้นจาก 1 แต่ในโค้ดพื้นฐาน อาร์เรย์จะถูกจัดทำดัชนีเริ่มต้นจาก 0 ดังนั้น ID จึงถูก map โดยการลบ 1 ออก

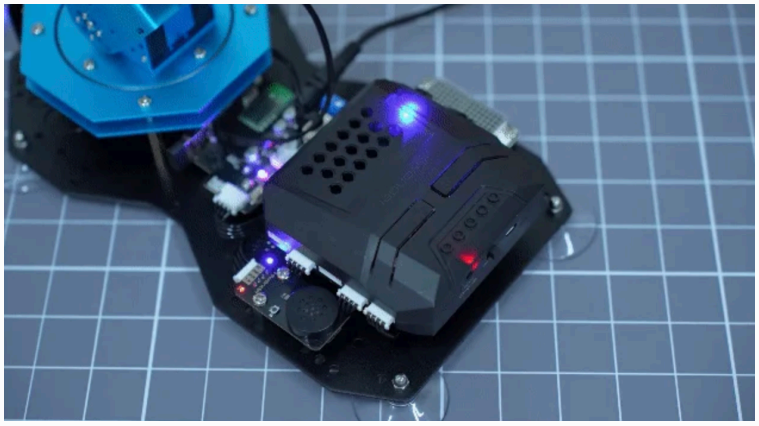
ถัดไป ตัวควบคุมจะตั้งค่าไฟ RGB ที่มี ID 1 เป็น 0:0:0 และตั้งค่าไฟ RGB ที่มี ID 2 ให้แสดงสีเขียวโดยตั้งค่า RGB เป็น 0:255:0 จากนั้นจะเรียกฟังก์ชันเขียนอีกครั้งเพื่อส่งข้อมูล ปิดไฟสีแดงและเปิดไฟสีเขียวสำหรับไฟ RGB 2 ทำเช่นเดียยวกันกับสีน้ำเงิน ทำจนครบทั้ง 6 ดวง

```
18    while True:
19      Hiwonder.Neopixel_onboard.setItem(1-1,255,0,0)
20      Hiwonder.Neopixel_onboard.write()
21      time.sleep(delay_time)
22      Hiwonder.Neopixel_onboard.setItem(1-1,0,0,0)
23      Hiwonder.Neopixel_onboard.write()
24      Hiwonder.Neopixel_onboard.setItem(2-1,0,255,0)
25      Hiwonder.Neopixel_onboard.write()
26      time.sleep(delay_time)
27      Hiwonder.Neopixel_onboard.setItem(2-1,0,0,0)
28      Hiwonder.Neopixel_onboard.write()
29      Hiwonder.Neopixel_onboard.setItem(3-1,0,0,255)
30      Hiwonder.Neopixel_onboard.write()
31      time.sleep(delay_time)
32      Hiwonder.Neopixel_onboard.setItem(3-1,0,0,0)
33      Hiwonder.Neopixel_onboard.write()
34      Hiwonder.Neopixel_onboard.setItem(6-1,255,0,0)
35      Hiwonder.Neopixel_onboard.write()
36      time.sleep(delay_time)
37      Hiwonder.Neopixel_onboard.setItem(6-1,0,0,0)
38      Hiwonder.Neopixel_onboard.write()
39      Hiwonder.Neopixel_onboard.setItem(5-1,0,255,0)
40      Hiwonder.Neopixel_onboard.write()
41      time.sleep(delay_time)
42      Hiwonder.Neopixel_onboard.setItem(5-1,0,0,0)
43      Hiwonder.Neopixel_onboard.write()
44      Hiwonder.Neopixel_onboard.setItem(4-1,0,0,255)
45      Hiwonder.Neopixel_onboard.write()
46      time.sleep(delay_time)
47      Hiwonder.Neopixel_onboard.setItem(4-1,0,0,0)
48      Hiwonder.Neopixel_onboard.write()
49      time.sleep(delay_time)
50
51  Hiwonder.startMain(start_main)
```

กระบวนการนี้จะดำเนินต่อไปเป็นวงจร ทำซ้ำขั้นตอนการสลับไฟไปเรื่อยๆ

2.4. ผลลัพธ์

ไฟ RGB บนตัวควบคุม CoreX จะสว่างขึ้นและดับลงตามลำดับสีแดง สีเขียว และสีน้ำเงิน



3. การวัดระยะทางด้วยคลื่นอัลตราโซนิค

3.1. ข้อมูลอุปกรณ์

ส่วนต่อประสานการสื่อสารแบบ I2C ซึ่งช่วยให้สามารถอ่านระยะทางที่วัดได้จากเซ็นเซอร์อัลตราโซนิกผ่านการสื่อสารแบบ I2C ได้

นอกจากนี้ บริเวณหัววัดอัลตราโซนิกยังประกอบด้วยไฟ LED RGB สองดวง ซึ่งไม่เพียงแต่ช่วยให้ปรับความสว่างของแสงได้เท่านั้น แต่ยังช่วยให้สร้างสีสันต่างๆ ได้หลากหลาย การเปลี่ยนแปลงช่องสีแดง (R) สีเขียว (G) และสีน้ำเงิน (B) รวมถึงการสามารถผสมผสานของสีได้ด้วย

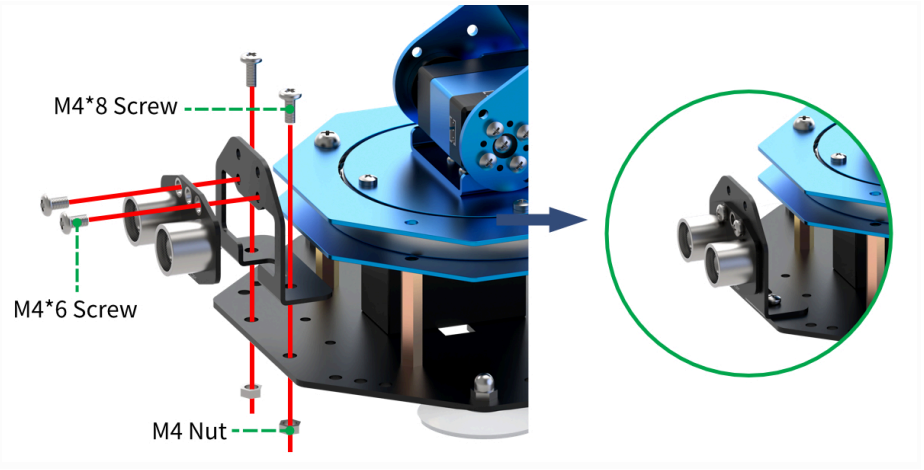


3.2. การเชื่อมต่อ

เชื่อมต่อ Dot Matrix เข้ากับพอร์ตหมายเลข 6 ของ CoreX

เชื่อมต่อ Ultrasonic Sensor เข้ากับพอร์ตหมายเลข 4 ของ CoreX





3.3. การวิเคราะห์โปรแกรม Python

(1) โหลดไลบรารี

ไลบรารี Hiwonder ประกอบด้วยไลบรารีเซ็นเซอร์ สัญญาณเตือนแรงดันต่ำ ฯลฯ
ไลบรารี Time ใช้สำหรับฟังก์ชันที่เกี่ยวข้องกับเวลา
ไลบรารี LSC จัดการการสื่อสารระดับต่ำระหว่างตัวควบคุมและบอร์ดควบคุมเซอร์โว และไลบรารี Buzzer ควบคุมเสียงกริ่ง

```
1  import Hiwonder
2  import time
3  from Hiwonder import LSC
4  from Hiwonder import Buzzer
```

(2) สร้าง Object Dot matrix LED และผูกเข้ากับพอร์ต IO 6 สร้าง Object โมดูลอัลตราโซนิกและผูกเข้ากับพอร์ต IIC 4 สร้างตัวแปร “ **ระยะทาง** ” เพื่อเก็บค่าระยะทางที่วัดได้จากโมดูลอัลตราโซนิก

```
6  # initialize variables
7  distance = 0
8  i2csonar_3 = Hiwonder.I2CSonar(Hiwonder.Port(4))
9  digitalTube_6 = Hiwonder.Digitaltube(Hiwonder.Port(6))
```

```
12 LSC.runActionGroup(0,1)
13
14
15
16
17 def start_main():
18     global _var_8ddd_79bb
19     global i2csonar_3
20     global digitalTube_6
```

ภายในฟังก์ชันหลัก Method getDistance ของโมดูลอัลตราโซนิกจะถูกเรียกใช้ในลูปเพื่ออ่านค่าระยะทาง ค่าที่ส่งกลับมาจะถูกปัดเศษโดยใช้เมธอด round และข้อมูลที่เกี่ยวข้องจะถูกพิมพ์ลงบนตำแหน่งที่ 1 ของเมทริกซ์ LED โดยใช้เมธอด showNum

```
while True:
    _var_8ddd_79bb = round(i2csonar_3.getDistance())
    digitalTube_6.showNum((_var_8ddd_79bb,1))
    if (_var_8ddd_79bb>10):
        Hiwonder.Neopixel_onboard.fill(0,255,0)
        i2csonar_3.setRGB(0,0,255,0)
    else:
        Hiwonder.Neopixel_onboard.fill(255,0,0)
        i2csonar_3.setRGB(0,255,0,0)
        Buzzer.playTone(1976,500,True)
    time.sleep(0.1)

Hiwonder.startMain(start_main)

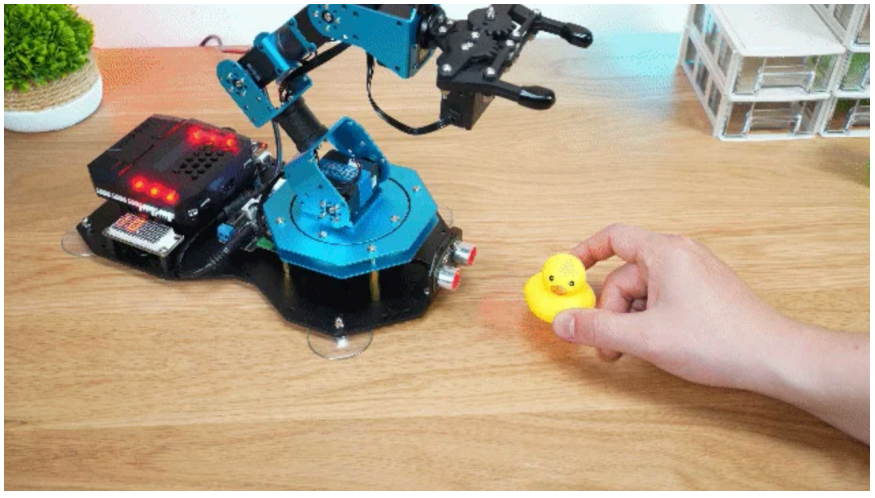
```

จากนั้นระบบจะตรวจสอบค่าระยะทาง หากระยะทางมากกว่าค่าเกณฑ์ 10 ซม. ไฟ RGB บนกล่อง CoreX จะเปลี่ยนเป็นสีเขียว (R:G:B = 0:255:0) โดยใช้เมธอด fill จากโมดูลไฟ RGB ของไลบรารี Hiwonder นอกจากนี้ยังเรียกใช้เมธอด setRGB ของโมดูลอัลตราโซนิกเรืองแสงเพื่อให้ไฟทั้งหมดเปลี่ยนเป็นสีเขียวด้วย

หากระยะห่างต่ำกว่าเกณฑ์ที่กำหนด ไฟ RGB บนกล่อง CoreX จะเปลี่ยนเป็นสีแดง (R:G:B = 255:0:0) และไฟทั้งหมดบนโมดูลอัลตราโซนิกเรืองแสงจะเปลี่ยนเป็นสีแดง นอกจากนี้ ระบบจะเรียกใช้เมธอด playTone จากไลบรารี Buzzer เพื่อเล่นเสียงที่มีความถี่ 1976 Hz ที่โน้ต B6 เป็นเวลา 500 มิลลิวินาที หรือครึ่งจังหวะ

3.4. ผลลัพธ์

ระยะทางที่ตรวจจับได้โดย Ultrasonic Sensor จะแสดงผลผ่าน Dot Matrix Display เมื่อระยะทางมากกว่า 10 ซม. ไฟ LED RGB ของอัลตราโซนิกจะสว่างเป็นสีเขียว หากน้อยกว่า จะแสดงสีแดง และจะมีเสียงดังขึ้น



4. การควบคุมด้วยปุ่มกด

4.1. ข้อมูลอุปกรณ์

ปุ่มกดบน CoreX เป็นส่วนประกอบอินพุตทั่วไปในระบบฝังตัวใช้สำหรับการโต้ตอบกับผู้ใช้และฟังก์ชัน โดยทั่วไปจะใช้เพื่อเริ่มต้น รีเซ็ต หรือเรียกใช้ฟังก์ชันเฉพาะ

4.2. การเชื่อมต่อ

ปุ่มกดบน A/B ถูกฝัง อยู่บน CoreX อยู่แล้ว



4.3. การวิเคราะห์โปรแกรม Python

(1) โหลดไลบรารี

ไลบรารี Hiwonder ประกอบด้วยไลบรารีเซ็นเซอร์ สัญญาณเตือนแรงดันต่ำ ฯลฯ
ไลบรารี Time ใช้สำหรับฟังก์ชันที่เกี่ยวข้องกับเวลา
และไลบรารี LSC จัดการการสื่อสารระดับต่ำระหว่างตัวควบคุมหลักและบอร์ดควบคุมเซอร์โว

```
1  import Hiwonder
2  import time
3  from Hiwonder import LSC
```

(2) หลังจากเปิดเครื่อง แขนหุ่นยนต์ ฟังก์ชันหลักจะทำงานโดยอัตโนมัติเพื่อเรียกใช้ Action Group ที่ 0 เพื่อวางแขนหุ่นยนต์ในสถานะเริ่มต้นพร้อมทำงาน

เมื่อตรวจพบว่ามีการกดปุ่ม A ฟังก์ชันตรรกะจะควบคุมแขนหุ่นยนต์ให้เรียกใช้ Action Group ที่ 3 หนึ่งครั้ง
เมื่อตรวจพบว่ามีการกดปุ่ม B ฟังก์ชันตรรกะจะควบคุมแขนหุ่นยนต์ให้เรียกใช้ Action Group ที่ 2 หนึ่งครั้ง

เพื่อป้องกันการรบกวนระหว่างตรรกะทั้งสามนี้ ตรรกะสำหรับการจัดการการกดปุ่ม A และ B จะมีผลก็ต่อเมื่อฟังก์ชัน actionFinish ตรวจสอบแล้วว่าบอร์ดควบคุมเซอร์โวอยู่ในสถานะว่าง และแขนหุ่นยนต์ไม่ได้กำลังดำเนินการกลุ่มคำสั่งใดๆ อยู่

```
6  def start_main():
7    LSC.runActionGroup(0,1)
8
9
10 def on_button_A_clicked():
11   if LSC.actionFinish():
12     LSC.runActionGroup(3,1)
13
14
15 def on_button_B_clicked():
16   if LSC.actionFinish():
17     LSC.runActionGroup(2,1)
```

(3) ภายในไลบรารี Hiwonder มีเธรดที่คอยตรวจสอบสถานะการกดปุ่ม ตรรกะที่แตกต่างกันจะถูกดำเนินการขึ้นอยู่กับว่ากดปุ่มใด โดยใช้ฟังก์ชัน Button_A.Clicked และ Button_B.Clicked

ฟังก์ชัน start_main() ต้องถูกเรียกใช้ในเธรดใหม่โดยใช้ Method startMain จากไลบรารี Hiwonder

```
19    Hiwonder.startMain(start_main)
```

```
20    Hiwonder.Button_A.Clicked(on_button_A_clicked)
21    Hiwonder.Button_B.Clicked(on_button_B_clicked)
```

4.4. ผลลัพธ์

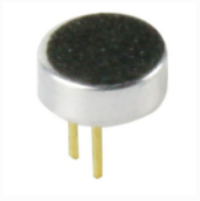
การกดปุ่ม A จะทำให้แขนหุ่นยนต์ทำการเคลื่อนไหวกลุ่มที่ 3 หนึ่งครั้ง (จับวัตถุไปทางซ้ายในมุม 45°)

การกดปุ่ม B จะทำให้แขนหุ่นยนต์ทำการเคลื่อนไหวกลุ่มที่ 2 หนึ่งครั้ง (จับวัตถุไปทางซ้ายในมุม 90°)

5. การควบคุมด้วยการตรวจจับเสียง

5.1. ข้อมูลอุปกรณ์

เซ็นเซอร์เสียงในตัวเป็นอุปกรณ์ที่ใช้ตรวจจับความดังของเสียงภายนอก สามารถอ่านค่าผ่านขา ADC เพื่อวัดระดับเสียง หลักการทำงานหลักคือการสั่นสะเทือนของไดอะแฟรมไมโครโฟนเนื่องจากเสียง ทำให้ฟิล์มไฟฟ้าสถิตภายในไมโครโฟนสั่น และส่งข้อมูลไปยังตัวควบคุม



5.2. การเชื่อมต่อ

Sensor ตรวจจับเสียงถูกฝัง อยู่บน CoreX อยู่แล้ว

5.3. การวิเคราะห์โปรแกรม Python

(1) โหลดไลบรารี

ไลบรารี Hiwonder ประกอบด้วยไลบรารีเซ็นเซอร์ สัญญาณเตือนแรงดันต่ำ ฯลฯ

ไลบรารี Time ใช้สำหรับฟังก์ชันที่เกี่ยวข้องกับเวลา

และไลบรารี kinematics จัดเตรียมอัลกอริธึมที่จำเป็นสำหรับการคำนวณการเคลื่อนที่

```
1    import Hiwonder
2    import time
3    import kinematics
```

เกี่ยวกับตัวแปรเริ่มต้น

ตัวแปร **Z_position** แทนตำแหน่งของปลายแขนหุ่นยนต์ตามแกน Z ในระบบพิกัดของแขนหุ่นยนต์

ส่วนตัวแปร **state** แทนทิศทางการเคลื่อนที่ของแขนหุ่นยนต์ เมื่อค่า*สถานะเป็น 0 แขนจะเคลื่อนที่ขึ้น* และเมื่อค่า*สถานะเป็น 1 แขนจะเคลื่อนที่ลง*

```
5   # initialize variables
6   Z_position = 0
7   state = 0
```

(2) เมื่อฟังก์ชันหลักเริ่มทำงาน set_link_length ของไลบรารี kinematics จะถูกเรียกใช้เพื่อกำหนดความยาวของแต่ละส่วนของแขนหุ่นยนต์สำหรับการคำนวณจลนศาสตร์ผกผัน จากนั้น ฟังก์ชัน ki_move จะถูกใช้เพื่อย้ายปลายแขนหุ่นยนต์ไปยังตำแหน่ง (0, 17, 20.5) ด้วยมุมเอียง 0° ในเวลา 1000 มิลลิวินาที ซึ่งเป็นการเริ่มต้นแขนหุ่นยนต์ไปยังตำแหน่งที่ต้องการ

ค่าแกน Z ปัจจุบันของปลายแขนหุ่นยนต์ (Z_position) จะต้องถูกตั้งค่าเป็น 20.5 พร้อมกัน และทิศทางการเคลื่อนที่ของแขนหุ่นยนต์จะต้องเป็น 0 ซึ่งจะทำให้แขนหุ่นยนต์เคลื่อนที่ขึ้นด้านบน

```
10  def start_main():
11    global Z_position
12    global state
13
14    kinematics.set_link_length(6.9,9.5,9.5,16.9)
15    kinematics.ki_move(0,17,20.5,0,1000)
16    Z_position = 20.5
17    state = 0
```

(3) ในลูปฟังก์ชันหลัก Method read จะอ่านค่า Sound_onboard จากไลบรารี Hiwonder จะถูกเรียกเพื่ออ่านระดับเสียงสิ่งแวดล้อมปัจจุบัน หากระดับเสียงสิ่งแวดล้อมเกิน 50 ระบบจะตรวจสอบทิศทางการเคลื่อนที่ปัจจุบันของแขนหุ่นยนต์ ซึ่งแสดงด้วยตัวแปร state ก่อน

หากค่า state เป็น 0 หมายความว่าแขนหุ่นยนต์กำลังเคลื่อนที่ขึ้น ในกรณีนี้ตำแหน่งแกน Z ของปลายแขนหุ่นยนต์ ซึ่งแสดงด้วย Z_position จะเพิ่มขึ้น 1 หากค่า Z_position ที่อัปเดตแล้วเกินความสูงสูงสุดที่อนุญาตคือ 25.8 ค่าจะถูกจำกัดไว้ที่ 25.8 และค่า state จะถูกตั้งค่าเป็น 1 ในกรณีนี้ แขนหุ่นยนต์จะเคลื่อนที่ไปในทิศทางตรงกันข้ามในการวนซ้ำลูปครั้งถัดไป

เมื่อค่าสถานะเป็น 1 จะใช้ตรรกะเดียวกัน

```
18    while True:
19      if (Hiwonder.Sound_onboard.read()>50):
20        if (state==0):
21          Z_position+=1
22          if (Z_position>25.8):
23            Z_position = 25.8
24            state = 1
25        else:
26          Z_position+=-1
27          if (Z_position<16.4):
28            Z_position = 16.4
29            state = 0
30        kinematics.ki_move(0,17,Z_position,0,500)
```

```
32  Hiwonder.startMain(start_main)
```

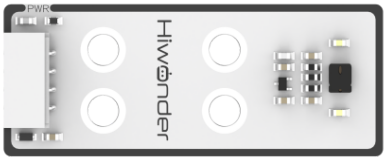
5.4. ผลลัพธ์

เมื่อตรวจพบความเข้มของเสียงมากกว่า 50 แขนหุ่นยนต์จะเคลื่อนที่ขึ้นด้านบนก่อน หลังจากถึงตำแหน่งจำกัดแล้วจึงจะเคลื่อนที่ลงด้านล่าง (หากต้องการให้ใช้เสียงมากกว่านี้ ในการออกคำสั่งให้แขนขยับ ลองให้นักเรียนเปลี่ยนค่า Sound_onboard.read()> ดูครับ)

6. การจดจำสี

6.1. ข้อมูลอุปกรณ์

เซ็นเซอร์ตรวจจับสี สามารถรับรู้สีของวัตถุที่วัด ความเข้มของแสงโดยรอบ การตรวจจับระยะใกล้ของวัตถุ และการตรวจจับท่าทางแบบไม่สัมผัส รวมถึงฟังก์ชันอื่นๆ เซ็นเซอร์นี้ผสานรวมการตรวจจับสี RGB ทำให้สามารถระบุสีของวัตถุต่างๆ ได้ นอกจากนี้ยังผสานรวมการตรวจจับแสงโดยรอบ



6.2. การเชื่อมต่อ



เชื่อมต่อ เซ็นเซอร์ตรวจจับสี เข้ากับพอร์ตหมายเลข 5 ของคอนโทรลเลอร์ CoreX และ
เชื่อมต่อ Dot Matrix Display เข้ากับพอร์ตหมายเลข 6 ของคอนโทรลเลอร์ CoreX

6.3. การวิเคราะห์โปรแกรม Python

(1) โหลดไลบรารี

ไลบรารี Hiwonder ประกอบด้วยไลบรารีเซ็นเซอร์ สัญญาณเตือนแรงดันต่ำ ฯลฯ
ไลบรารี Time ใช้สำหรับฟังก์ชันที่เกี่ยวข้องกับเวลา
และไลบรารี LSC จัดการการสื่อสารระดับต่ำระหว่างตัวควบคุมและบอร์ดควบคุมเซอร์โว

```
1  import Hiwonder
2  import time
3  from Hiwonder import LSC
```

(2) สร้าง Object โมดูลดอทเมทริกซ์ digitalTube_6 และผูกเข้ากับพอร์ต 6 และ
สร้าง Object โมดูลเซ็นเซอร์สี colorsensor_4 และผูกเข้ากับพอร์ต 4

```
5  # initialize variables
6  digitalTube_6 = Hiwonder.Digitaltube(Hiwonder.Port(6))
7  colorsensor_4 = Hiwonder.ColorSensor(Hiwonder.Port(4))
```

เมื่อเปิดใช้งานแขนหุ่นยนต์ ระบบจะเรียกใช้ฟังก์ชันหลักเป็นค่าเริ่มต้น ซึ่งจะรัน
กลุ่มการทำงานที่ 0 หนึ่งครั้งเพื่อวางแขนหุ่นยนต์ในตำแหน่งเริ่มต้น จากนั้นจะเรียกใช้
ฟังก์ชัน sleep จากไลบรารีเวลาเพื่อให้ระบบเข้าสู่โหมดพักการทำงานเป็นเวลา 1 วินาที

```
10 def start_main():
11   global digitalTube_6
12   global colorsensor_4
13
14   LSC.runActionGroup(0,1)
15   time.sleep(1)
16
```

เริ่มต้นของการวนซ้ำแต่ละครั้งในลูปฟังก์ชันหลัก Method fill ของ Object
Neopixel_onboard จากโมดูล CoreX ของไลบรารี Hiwonder จะถูกเรียกเพื่อปิดไฟ LED
RGB ทั้งหมดบนบอร์ด (R:G:B = 0:0:0) จากนั้นจะ drawBitMap ของ Object Dot Matrix
LED จะถูกเรียกเพื่อปิดไฟ LED ทั้งหมดบน Dot Matrix เพื่อล้างหน้าจอ

จากนั้น จะเรียก Method readColorName ของ Object เซ็นเซอร์สีเพื่ออ่าน
ผลลัพธ์การจดจำสี

ถ้าค่าที่ส่งกลับมาเป็น 2 แสดงว่าตรวจพบสีแดง ในกรณีนี้ Method fill ของ
Object CoreX box RGB light array จะถูกเรียกอีกครั้งเพื่อเปลี่ยนไฟ RGB ทั้งหมดให้
เป็นสีแดง (R:G:B = 255:0:0) จากนั้น Method drawStr ของ Object matrix จะถูกเรียก
เพื่อแสดงตัวอักษรตัวแรกของชื่อสีที่ตำแหน่ง (6,0) บน matrix และระบบจะเข้าสู่โหมด
พักเครื่องเป็นเวลา 2 วินาที

มีการดำเนินการที่คล้ายกันสำหรับค่าส่งคืนอื่นๆ

```
  while True:
    Hiwonder.Neopixel_onboard.fill(0,0,0)
    digitalTube_6.drawBitMap((0x0,0x0,0x0,0x0,0x0,0x0,0x0,0x0,0x0,0x0,0x0,0x0,0x0,0x0,0x0,0x0))
    if colorsensor_4.readColorName() == 2:
      Hiwonder.Neopixel_onboard.fill(255,0,0)
      digitalTube_6.drawStr((6,0,'R'))
      time.sleep(2)
    else:
      if colorsensor_4.readColorName() == 5:
        Hiwonder.Neopixel_onboard.fill(0,255,0)
        digitalTube_6.drawStr((6,0,'G'))
        time.sleep(2)
      else:
        if colorsensor_4.readColorName() == 7:
          Hiwonder.Neopixel_onboard.fill(0,0,255)
          digitalTube_6.drawStr((6,0,'B'))
          time.sleep(2)
        else:
          if colorsensor_4.readColorName() == 4:
            Hiwonder.Neopixel_onboard.fill(255,255,0)
            digitalTube_6.drawStr((6,0,'Y'))
            time.sleep(2)
```

```
Hiwonder.startMain(start_main)
```

6.4. ผลลัพธ์

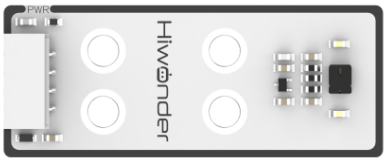
เมื่อตรวจพบสีแดง จอแสดงผล RGB จะแสดงแสงสีแดง และDot Matrix LED จะแสดง " R "
เมื่อตรวจพบสีเขียว ไฟแสดงสถานะ RGB จะเป็นสีเขียว และDot Matrix LED จะแสดง “ G ”
เมื่อตรวจพบสีน้ำเงิน จอแสดงผล RGB จะแสดงแสงสีน้ำเงิน และDot Matrix LED จะแสดง " B "
เมื่อตรวจพบสีเหลือง วงจร RGB จะแสดงแสงสีเหลือง และDot Matrix LED จะแสดง “ Y ”

7. การจัดเรียงบล็อคสี

7.1. ข้อมูลอุปกรณ์

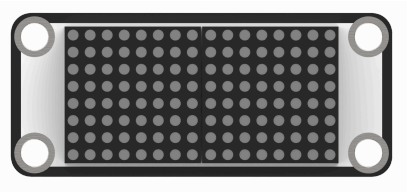
(1) เซ็นเซอร์สี

เซ็นเซอร์นี้สามารถรับรู้สีของวัตถุที่วัด ความเข้มของแสงโดยรอบ การตรวจจับระยะใกล้ของวัตถุ และการตรวจจับท่าทางแบบไม่สัมผัส รวมถึงฟังก์ชันอื่นๆ เซ็นเซอร์นี้ผสานรวมการตรวจจับสี RGB ทำให้สามารถระบุสีของวัตถุต่างๆ ได้ นอกจากนี้ยังผสานรวมการตรวจจับแสงโดยรอบ ทำให้สามารถวัดความเข้มของแสงภายใต้สภาพแสงที่แตกต่างกันได้ ด้วย LED อินฟราเรดในตัว จึงสามารถใช้สำหรับการตรวจจับระยะใกล้ของวัตถุได้



(2) โมดูลดอทเมทริกซ์

โมดูลดอทเมทริกซ์ LED ใช้หน้าจอแสดงผล LED ที่มีความสว่างสูง แสดงผลได้โดยไม่กระพริบ และต่อสายได้ง่าย สามารถแสดงตัวเลข ข้อความ รูปแบบ และเนื้อหาอื่นๆ ได้ โมดูลนี้ประกอบด้วยไฟ LED สีแดงขนาด 8x8 สองดวง



7.2. การเชื่อมต่อ

เชื่อมต่อ เซ็นเซอร์ตรวจจับสี เข้ากับพอร์ตหมายเลข 5 ของคอนโทรลเลอร์ CoreX และ
เชื่อมต่อ Dot Matrix Display เข้ากับพอร์ตหมายเลข 6 ของคอนโทรลเลอร์ CoreX

7.3. การวิเคราะห์โปรแกรม Python

(1) โหลดไลบรารี ไลบรารี Hiwonder ประกอบด้วยไลบรารีเซ็นเซอร์ การแจ้งเตือนแรงดันต่ำ ฯลฯ ไลบรารีเวลาใช้สำหรับฟังก์ชันที่เกี่ยวข้องกับเวลา ไลบรารีจลนศาสตร์ใช้สำหรับอัลกอริธึมจลนศาสตร์ และไลบรารี LSC ใช้สำหรับการสื่อสารระดับต่ำระหว่างตัวควบคุมและบอร์ดควบคุมเซอร์โว

```
1  import Hiwonder
2  import time
3  import kinematics
4  from Hiwonder import LSC
```

(2) สร้าง Object โมดูลดอทเมทริกซ์ digitalTube_6 และผูกเข้ากับพอร์ต 6 และสร้าง Object โมดูลเซ็นเซอร์สี colorsensor_4 และผูกเข้ากับพอร์ต 4

```
6  # initialize variables
7  digitalTube_6 = Hiwonder.Digitaltube(Hiwonder.Port(6))
8  colorsensor_4 = Hiwonder.ColorSensor(Hiwonder.Port(4))
```

เมื่อเปิดใช้งาน ระบบจะเรียกใช้ฟังก์ชันหลักโดยอัตโนมัติ ในขั้นต้น ระบบจะใช้ Method set_link_length และ ki_move จากไลบรารี kinematics เพื่อกำหนดความยาวของแต่ละส่วนของแขนและเคลื่อนแขนไปยังตำแหน่งเริ่มต้น จากนั้น ระบบจะเรียกฟังก์ชัน moveServo จากไลบรารี LSC เพื่อเคลื่อน Servo 1 ไปยังตำแหน่ง 100 ซึ่งจะเปิดตัวจับยึด(Gripper)ของหุ่นยนต์ จากนั้นจะเรียกฟังก์ชัน sleep จากไลบรารี time เพื่อให้ระบบเข้าสู่โหมดพักเป็นเวลา 1 วินาที

```
11  def start_main():
12    global digitalTube_6
13    global colorsensor_4
14
15    kinematics.set_link_length(6.9,9.5,9.5,16.9)
16    kinematics.ki_move(-18,0,8,-32,1000)
17    LSC.moveServo(1,100,500)
18    time.sleep(1)
```

ในตอนเริ่มต้นของการวนซ้ำแต่ละครั้งในลูปฟังก์ชันหลัก เมธอด fill ของ Object Neopixel_onboard จากโมดูล CoreX ของไลบรารี Hiwonder จะถูกเรียกเพื่อปิดไฟ LED RGB ทั้งหมดบนบอร์ด (R:G:B = 0:0:0) จากนั้นเมธอด drawBitMap ของ Object Dot matrix LED จะถูกเรียกเพื่อปิดไฟ LED ทั้งหมดบนเมทริกซ์เพื่อล้างหน้าจอ

จากนั้น จะเรียก Method readColorName ของ Object เซ็นเซอร์สีเพื่ออ่านผลการจดจำ หากเซ็นเซอร์ตรวจพบสีแดง จะมีการดำเนินการดังต่อไปนี้ การจัดการค่าส่งคืนอื่นๆ เป็นไปตามตรรกะเดียวกัน

ถ้าค่าที่ส่งกลับมาเป็น 2 แสดงว่าตรวจพบสีแดง ในกรณีนี้ Method fill ของ Object CoreX box RGB light array จะถูกเรียกอีกครั้งเพื่อเปลี่ยนไฟ RGB ทั้งหมดให้เป็นสีแดง (R:G:B = 255:0:0) จากนั้นเมธอด drawStr ของ Object matrix จะถูกเรียกเพื่อแสดงตัวอักษรตัวแรกของชื่อสี(R)ที่ตำแหน่ง (6,0) บน matrix และระบบจะเข้าสู่โหมดพักเครื่องเป็นเวลา 2 วินาที

ถัดไป เซอร์โว 1 จะถูกย้ายไปยังตำแหน่ง 500 โดยใช้ฟังก์ชัน moveServo ซึ่งจะทำให้แขนหุ่นยนต์ปิดลงเพื่อจับบล็อกสี จากนั้น แขนหุ่นยนต์จะยกขึ้นและหมุน 90 องศาไปทางซ้ายโดยใช้วิธี ki_move จากไลบรารีจลศาสตร์ และวางบล็อกลง

หลังจากวางบล็อกแล้ว เซอร์โว 1 จะเคลื่อนกลับไปยังตำแหน่ง 100 เพื่อเปิดตัวจับยึดและปล่อยบล็อก สุดท้าย แขนหุ่นยนต์จะยกขึ้นอีกครั้งในทิศทางปัจจุบันและกลับไปยังตำแหน่งเดิมโดยใช้ฟังก์ชัน ki_move

```
while True:
    Hiwonder.Neopixel_onboard.fill(0,0,0)
    digitalTube_6.drawBitMap((0x0,0x0,0x0,0x0,0x0,0x0,0x0,0x0,0x0,0x0,0x0,0x0,0x0,0x0,0x0,0x0,0x0,0x0
    if colorsensor_4.readColorName() == 2:
        Hiwonder.Neopixel_onboard.fill(255,0,0)
        digitalTube_6.drawStr((6,0,'R'))
        time.sleep(2)
        LSC.moveServo(1,500,500)
        time.sleep(1)
        kinematics.ki_move(-17,0,20.5,0,800)
        time.sleep(0.8)
        kinematics.ki_move(-19.5,0,2.8,-60,800)
        time.sleep(0.8)
        LSC.moveServo(1,100,500)
        time.sleep(0.8)
        kinematics.ki_move(-17,0,20.5,0,800)
        time.sleep(0.8)
        kinematics.ki_move(-18,0,8,-32,1000)
        time.sleep(1)
```

ในกรณีสีอื่นๆ ก็จะใช้หลักการทำงานเดียวกัน

สีเขียว

```
else:
    if colorsensor_4.readColorName() == 5:
        Hiwonder.Neopixel_onboard.fill(0,255,0)
        digitalTube_6.drawStr((6,0,'G'))
        time.sleep(2)
        LSC.moveServo(1,500,500)
        time.sleep(1)
        kinematics.ki_move(-17,0,20.5,0,800)
        time.sleep(0.8)
        kinematics.ki_move(-12,12,20.5,0,800)
        time.sleep(0.8)
        kinematics.ki_move(-14,14,2.9,-59,800)
        time.sleep(0.8)
        LSC.moveServo(1,100,500)
        time.sleep(1)
        kinematics.ki_move(-12,12,20.5,0,800)
        time.sleep(0.8)
        kinematics.ki_move(-17,0,20.5,0,800)
        time.sleep(0.8)
        kinematics.ki_move(-18,0,8,-32,1000)
        time.sleep(1)
```

สีฟ้า

```
    else:
        if colorsensor_4.readColorName() == 7:
            Hiwonder.Neopixel_onboard.fill(0,0,255)
            digitalTube_6.drawStr((6,0,'B'))
            time.sleep(2)
            LSC.moveServo(1,500,500)
            time.sleep(1)
            kinematics.ki_move(-17,0,20.5,0,800)
            time.sleep(0.8)
            kinematics.ki_move(12,12,20.5,0,1000)
            time.sleep(1)
            kinematics.ki_move(14,14,2.9,-59,800)
            time.sleep(0.8)
            LSC.moveServo(1,100,500)
            time.sleep(1)
            kinematics.ki_move(12,12,20.5,0,1000)
            time.sleep(1)
            kinematics.ki_move(-17,0,20.5,0,1500)
            time.sleep(1.5)
            kinematics.ki_move(-18,0,8,-32,1000)
            time.sleep(1)
```

สีเหลือง

```
80          else:
81              if colorsensor_4.readColorName() == 4:
82                  Hiwonder.Neopixel_onboard.fill(255,255,0)
83                  digitalTube_6.drawStr((6,0,'Y'))
84                  time.sleep(2)
85                  LSC.moveServo(1,500,500)
86                  time.sleep(1)
87                  kinematics.ki_move(-17,0,20.5,0,800)
88                  time.sleep(0.8)
89                  kinematics.ki_move(17,0,20.5,0,1500)
90                  time.sleep(1)
91                  kinematics.ki_move(19.5,0,2.8,-60,800)
92                  time.sleep(0.8)
93                  LSC.moveServo(1,100,500)
94                  time.sleep(1)
95                  kinematics.ki_move(17,0,20.5,0,800)
96                  time.sleep(0.8)
97                  kinematics.ki_move(-17,0,20.5,0,1500)
98                  time.sleep(1)
99                  kinematics.ki_move(-18,0,8,-32,1000)
100                 time.sleep(1)
101
102 Hiwonder.startMain(start_main)
```

ข้อสอบก่อน-หลังเรียน

คำชี้แจง: ให้นักเรียนเลือกคำตอบที่ถูกต้องที่สุดเพียงข้อเดียว

---

1. Dot Matrix คืออะไร

ก. เซ็นเซอร์วัดระยะทาง
ข. จอแสดงผลแบบ LED ✓
ค. ปุ่มกด
ง. ไฟ RGB

2. RGB ย่อมาจากอะไร

ก. Red, Green, Blue (แดง, เขียว, น้ำเงิน) ✓
ข. Red, Gray, Black
ค. Robot, Game, Button
ง. Read, Get, Build

3. เซ็นเซอร์อัลตราโซนิกใช้วัดอะไร

ก. อุณหภูมิ
ข. ความชื้น
ค. ระยะทาง ✓
ง. สี

4. CoreX มีปุ่มกดกี่ปุ่ม

ก. 1 ปุ่ม
ข. 2 ปุ่ม (A และ B) ✓
ค. 3 ปุ่ม
ง. 4 ปุ่ม

5. WonderCam ใช้ทำอะไร

ก. วัดระยะทาง
ข. จดจำสีและรูปภาพ ✓
ค. เล่นเพลง
ง. วัดอุณหภูมิ

6. ไฟ RGB สามารถเปลี่ยนเป็นสีอะไรได้บ้าง

ก. แค่สีแดง
ข. แค่สีเขียว
ค. ได้หลายสี (ผสมสี) ✓
ง. ไม่สามารถเปลี่ยนสีได้

7. ถ้าต้องการให้ไฟ RGB กระพริบ ต้องใช้คำสั่งอะไรช่วย

ก. print()
ข. time.sleep() ✓
ค. input()
ง. move()

8. เซ็นเซอร์สีสามารถแยกแยะสีอะไรได้

ก. แดง, เขียว, น้ำเงิน ✓
ข. แค่สีดำ-ขาว
ค. ทุกสีที่มนุษย์มองเห็น
ง. ไม่สามารถแยกสีได้

9. การจับและวางบล็อคตามสี ต้องใช้อุปกรณ์อะไรบ้าง

ก. เฉพาะมือจับ
ข. มือจับ + เซ็นเซอร์สี ✓
ค. เฉพาะเซ็นเซอร์สี
ง. ปุ่มกด

10. ข้อใดเป็นลำดับขั้นตอนที่ถูกต้องในการจับบล็อคตามสี

ก. จับ → ตรวจสี → วาง
ข. ตรวจสี → จับ → วาง ✓
ค. วาง → จับ → ตรวจสี
ง. ตรวจสี → วาง → จับ